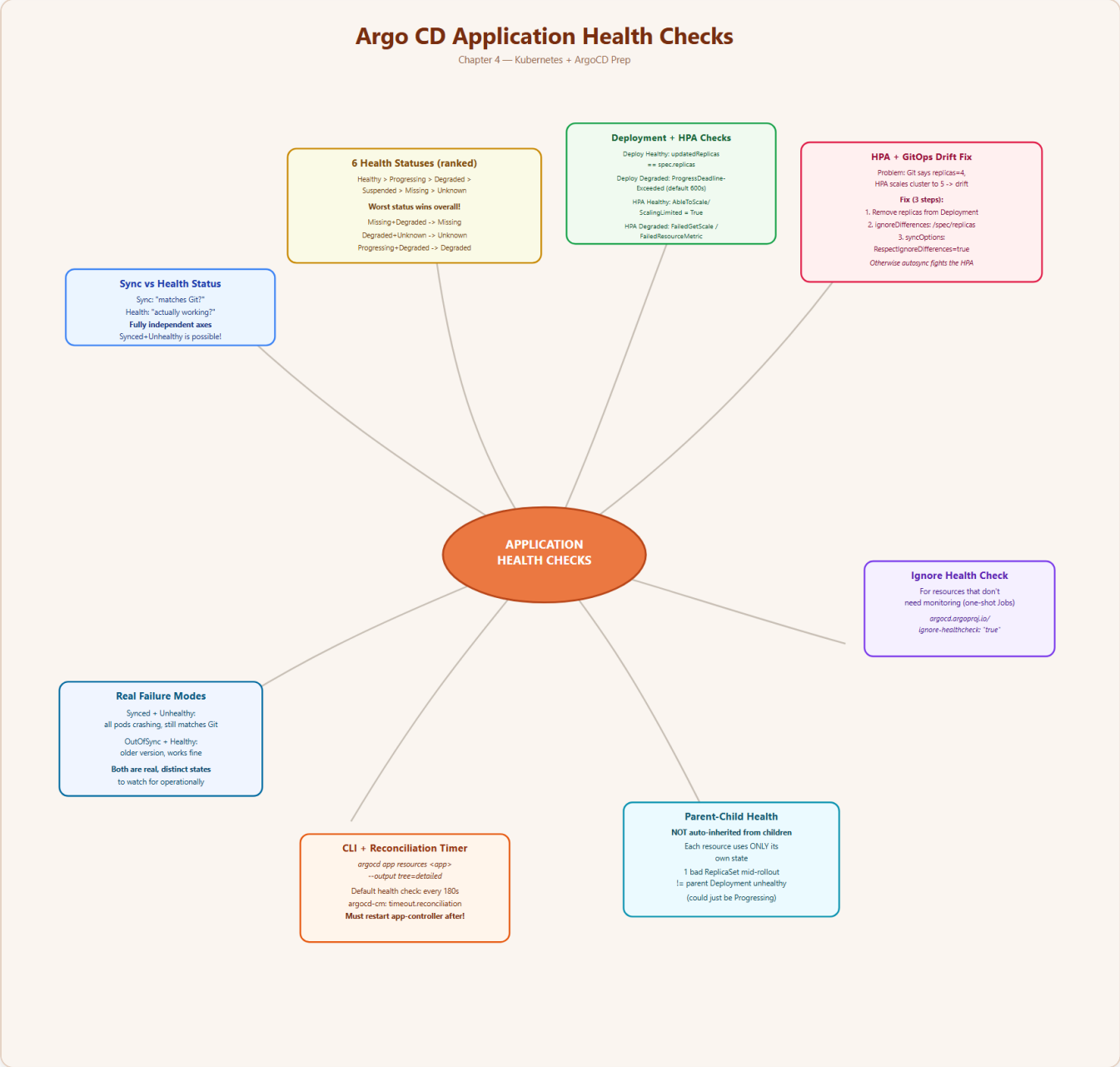
ARGO CD GITOPS › MY LEARNING NOTES › SESSION 04

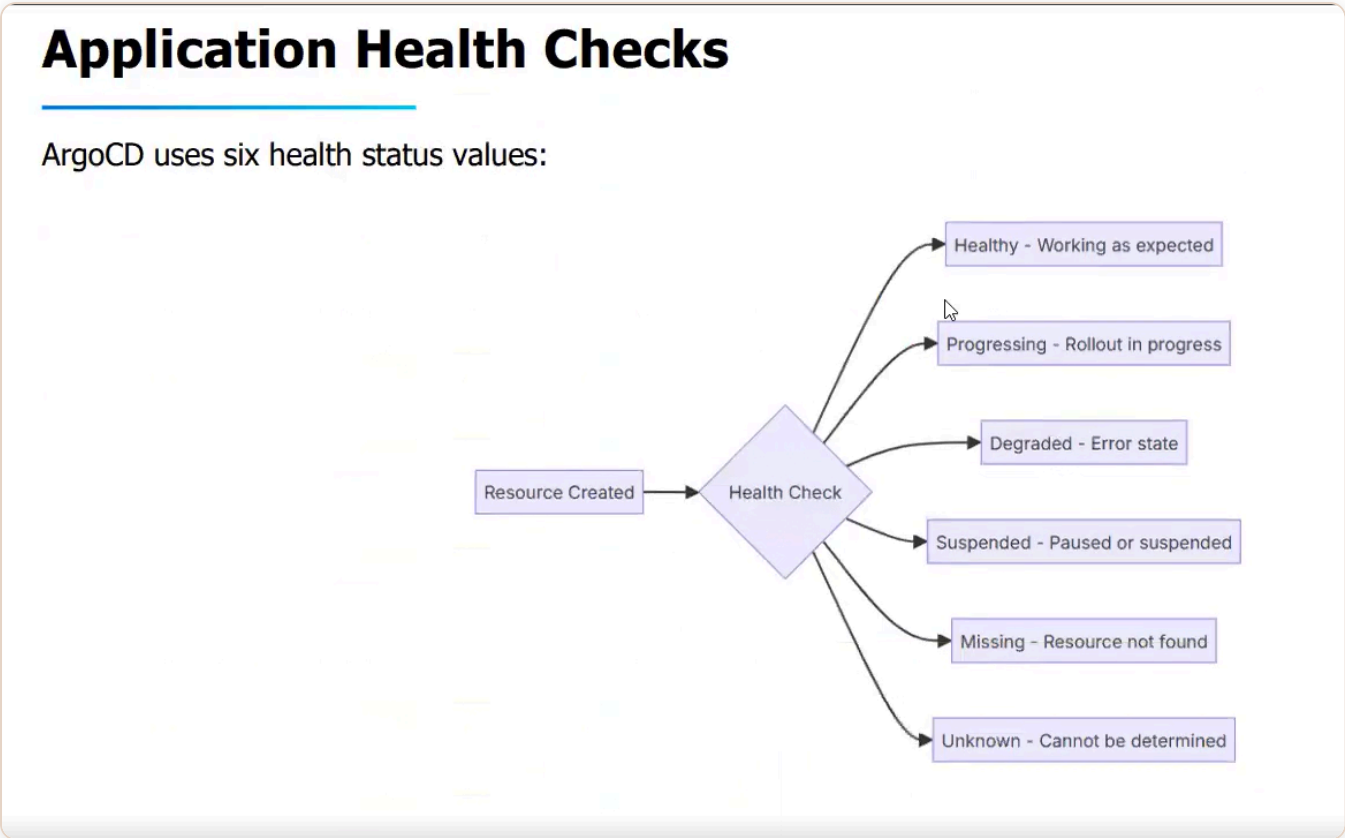
Argo CD Application Health Checks

Topic: Health vs Sync, HPA drift, Parent-Child | Chapter 4 | 2026-09-12

Quick Recall — Mind Map



Sync Status vs. Health Status — the Distinction That Matters Most



- **Sync Status** answers: "*Does the cluster match my Git repo?*"
- **Health Status** answers: "*Is the live code actually working?*"

These are **completely independent** of each other, and understanding the difference is critical for operating GitOps workflows:

- An app can be **perfectly Synced but Unhealthy** — all Pods are crashing, even though what's running is exactly what Git says should be running.
- An app can be **OutOfSync but Healthy** — running an older version that works perfectly fine, just not the latest Git commit.

The 6 Argo CD Health Status Values



Status	Meaning
Healthy	Working/operating as expected
Progressing	Rollout in progress
Degraded	Error state
Suspended	Paused or suspended
Missing	Resource not found
Unknown	Cannot be determined

The Worst-Case Aggregation Rule (Important, Non-Obvious)

The lower a status appears in this list, the worse it is. If an application has multiple resources each with their own status, the **lowest one in the list becomes the app's overall health** — one bad resource is enough to drag the whole application's reported health down.

Ranking from best to worst:

Healthy → **Progressing** → **Degraded** → **Suspended** → **Missing** →

Unknown

Worked examples:

- `Missing` + `Degraded` → app health = **Missing** (Missing is lower in the list than Degraded).
- `Degraded` + `Unknown` → app health = **Unknown** (Unknown is the lowest of all).
- `Progressing` + `Degraded` → app health = **Degraded**.

Built-In Health Checks for Specific Kubernetes Types

Deployment Health Check

Argo CD evaluates Deployments by checking **replica status**:

State	Condition
Healthy	`status.updatedReplicas` == `spec.replicas`; `status.availableReplicas` == `status.updatedReplicas`; no old ReplicaSets with running pods; generation matches observed generation
Progressing	Replicas being updated (rolling update); new pods starting but not yet available; rollout hasn't exceeded `progressDeadlineSeconds`
Degraded	Condition reason `ProgressDeadlineExceeded` (Kubernetes default: 600 seconds)

HorizontalPodAutoscaler (HPA) Health Check

State	Condition
Healthy	`AbleToScale` or `ScalingLimited` condition, status `True`
Progressing	Argo CD waiting for autoscaling conditions
Degraded	`AbleToScale` reason `FailedGetScale` / `FailedUpdateScale`, or `ScalingActive` reason `FailedGetResourceMetric` / `InvalidSelector`

⚠ The Special Case: Deployment + HPA Together (Replica-Count Drift)

The problem, concretely: a Deployment YAML sets `replicas: 4`. An HPA is also configured (e.g. scale up when CPU > 80%). CPU spikes, the HPA scales the Deployment to 5 pods.

- **Git (source of truth) says:** replicas = 4
- **Live cluster says:** replicas = 5 (HPA's doing)
- **Mismatch** — Argo CD would normally see this as drift.

The fix — don't fight the HPA, tell Argo CD to ignore that one field

1. **Remove `replicas` entirely from the Deployment spec** — let the HPA own scaling via its own `minReplicas` / `maxReplicas`.
2. **Tell the Application to ignore that field** via `ignoreDifferences`.
3. **Make automated syncs respect the ignore rule** via `RespectIgnoreDifferences=true`.

YAML

```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: argolab-hpa
  namespace: production
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: argolab
  minReplicas: 2
  maxReplicas: 10
```

YAML

```yaml
# In the Deployment itself:
spec:
  # IMPORTANT: 'replicas' is completely removed.
  # This prevents GitOps drift detection from triggering on scale events.
  selector:
    matchLabels: { app: argoapp }
  template:
    metadata: { labels: { app: argoapp } }
    spec:
      containers:
        - name: boxone
          image: nginx:1.27
```

YAML

```yaml
spec:
  ignoreDifferences:   # Ignore the dynamic replica count managed by the HPA controller
    - group: apps
      kind: Deployment
      jsonPointers:
        - /spec/replicas
```

YAML

```yaml
syncPolicy:
  automated:
    prune: true
    selfHeal: true
  syncOptions:
    - CreateNamespace=true
    - RespectIgnoreDifferences=true    # IMP: don't overwrite ignored fields during automated
syncs
```

Without `RespectIgnoreDifferences=true`, an automated sync could still stomp the HPA's live replica count back down to whatever (or nothing) is in Git, fighting the autoscaler.

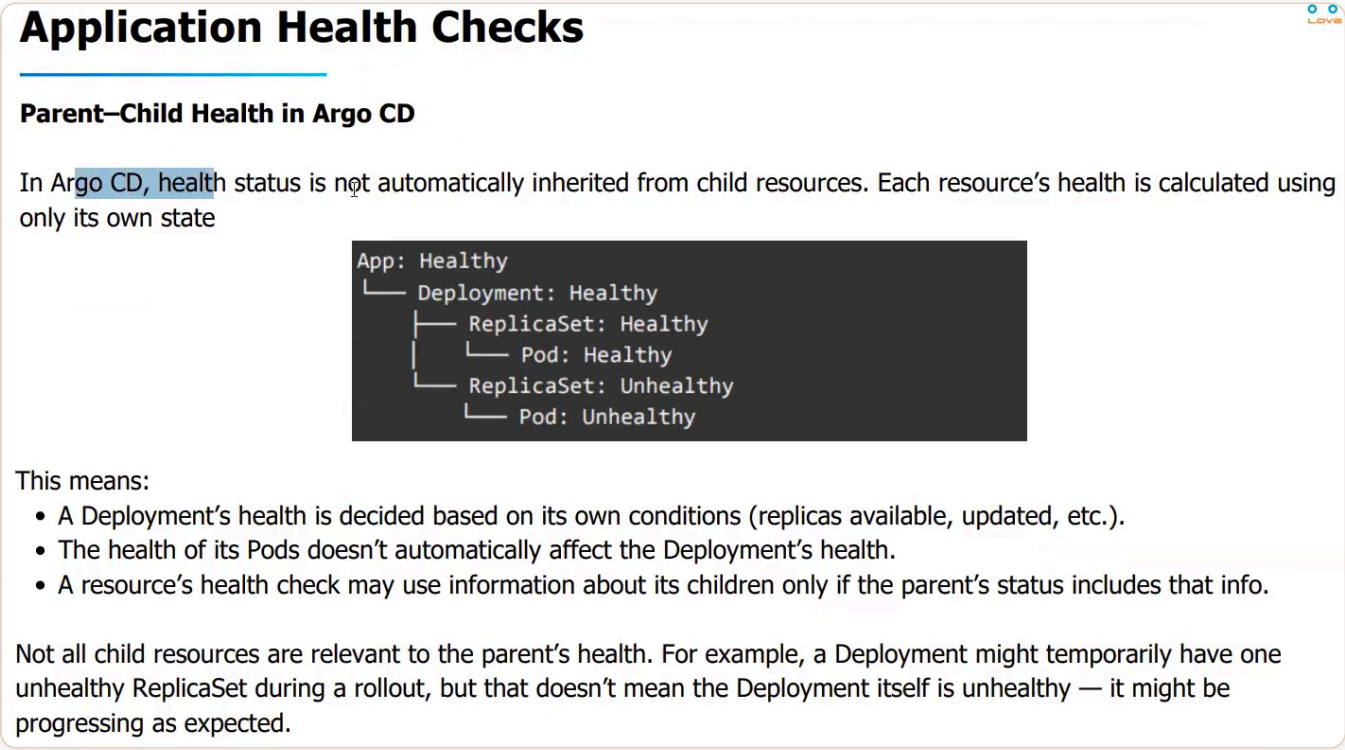
Ignoring Health Checks Entirely for a Resource

Some resources genuinely don't need ongoing health monitoring — a classic example: a one-shot `Job` that runs once and exits.

```yaml
metadata:
  annotations:
    argocd.argoproj.io/ignore-healthcheck: "true"
```

Parent–Child Health — NOT Automatically Inherited

In Argo CD, health status is not automatically inherited from child resources. Each resource's health is calculated using **only its own state**:



What this means in practice:

- A Deployment's health is decided based on **its own conditions** (replicas available, updated, etc.) — not by directly inheriting from its Pods.
- The health of its Pods doesn't automatically affect the Deployment's reported health.
- A resource's health check may use information about its children only if the parent's own status object happens to surface that info.

- Not every child resource is relevant to the parent's health: a Deployment might temporarily have one unhealthy ReplicaSet mid-rollout, but that doesn't automatically make the Deployment itself unhealthy — it may correctly be `Progressing` instead.

Checking Status via CLI

```bash
argocd app resources newapp --output tree=detailed
# GROUP  KIND        NAMESPACE  NAME                           ORPHANED  AGE  HEALTH   REASON
#        ConfigMap   red        web-v1                         No        23h
#        Service     red        nginx-deployment-svc           No        24h  Healthy
# apps   Deployment  red        nginx-deployment               No        24h  Healthy
# apps   ReplicaSet  red        nginx-deployment-77bc6bd484    No        24h  Healthy
#        Pod         red        nginx-deployment-77bc6bd484-25lwg  No    24h  Healthy
```

> **Watch the exact syntax**
>
> The correct flag syntax is `--output tree=detailed` (an `=`, not a space or hyphen) — easy typo to make.

```bash
argocd app resources argolab-production --output tree=detailed
# GROUP        KIND                   NAMESPACE   NAME              HEALTH   REASON
# apps         Deployment             production  argolab           Healthy
# apps         ReplicaSet             production  argolab-7b9867486 Healthy
#              Pod                    production  argolab-...-rt285 Healthy
#              Pod                    production  argolab-...-w75qx Healthy
# autoscaling  HorizontalPodAutoscaler production  argolab-hpa       Healthy  recommended size
matches current size
```

The Reconciliation Timer (How Often Health Gets Re-Checked)

Argo CD evaluates the health of all resources inside an application on its periodic background check. **By default, this happens every 180 seconds (3 minutes).**

```bash
kubectl -n argocd edit cm argocd-cm
```

```yaml
data:
  timeout.reconciliation: "60s"
```

```bash
# save and exit, then:
kubectl -n argocd rollout restart statefulset argocd-application-controller
```

The controller restart step is not optional

Editing the ConfigMap alone doesn't pick up the new interval until the Application Controller (itself a StatefulSet, as covered in Chapter 1) is restarted.

What I Practiced

- [ ] Created an app that was both Synced and Healthy, and reasoned through what would make it "Synced but Unhealthy" or "OutOfSync but Healthy" as two genuinely different failure/non-failure modes.
- [ ] Walked all 6 health statuses and memorized the worst-case-wins ranking rather than treating them as an unordered set.
- [ ] Read the exact Deployment and HPA health-check conditions Argo CD checks under the hood.
- [ ] Built the fix for the classic HPA-vs-GitOps replica drift problem: removed `replicas` from the Deployment, added `ignoreDifferences` on `/spec/replicas`, and set `RespectIgnoreDifferences=true`.
- [ ] Excluded a one-shot Job from health evaluation via `ignore-healthcheck: "true"`.
- [ ] Confirmed health is calculated per-resource, not inherited top-down, using `argocd app resources <app> --output tree=detailed`.
- [ ] Changed the global reconciliation interval via `argocd-cm`'s `timeout.reconciliation` and restarted the `argocd-application-controller` StatefulSet to apply it.

Merged into the master reference notes: argocd-master-notes.md — new "4. Application Health Checks" section.